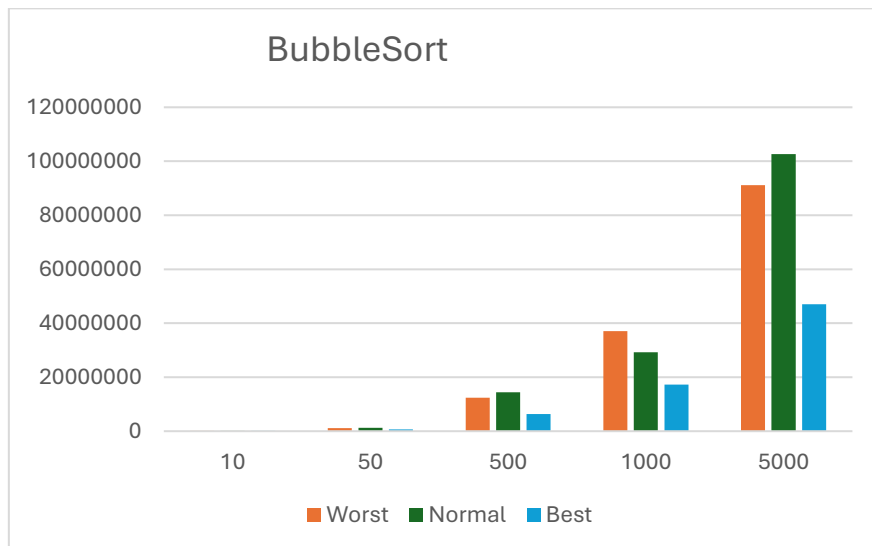


# Sorts

BubbleSort:

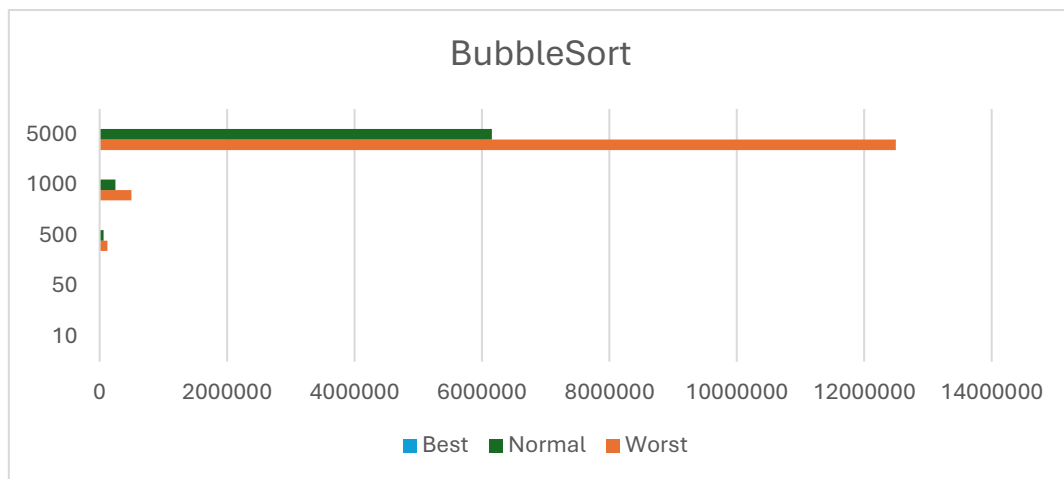
**Tiempos**(En Nanosegundos)

| Contenido | BubbleSort |           |          |
|-----------|------------|-----------|----------|
|           | Worst      | Normal    | Best     |
| 10        | 106000     | 48300     | 42499    |
| 50        | 1131300    | 1247800   | 651100   |
| 500       | 12350400   | 14440901  | 6399400  |
| 1000      | 37126600   | 29238701  | 17249400 |
| 5000      | 91168400   | 102650999 | 47059201 |



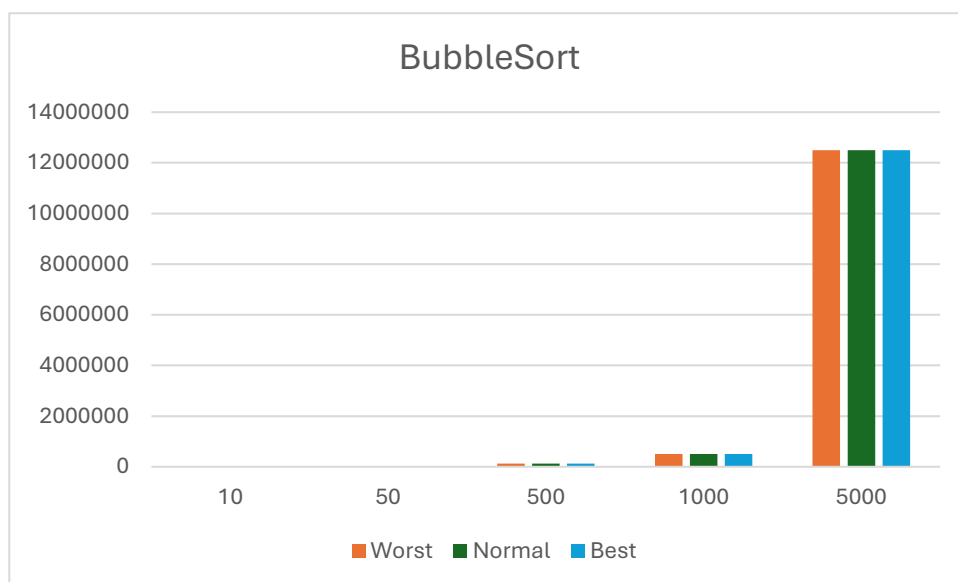
**Swaps**

| Contenido | Bubble Sort |         |      |
|-----------|-------------|---------|------|
|           | Worst       | Normal  | Best |
| 10        | 45          | 29      | 0    |
| 50        | 1225        | 656     | 0    |
| 500       | 124750      | 62029   | 0    |
| 1000      | 499500      | 248040  | 0    |
| 5000      | 12497500    | 6157252 | 0    |



## Comparaciones

| Contenido | Bubble   |          |          |
|-----------|----------|----------|----------|
|           | Worst    | Normal   | Best     |
| 10        | 45       | 45       | 45       |
| 50        | 1225     | 1225     | 1225     |
| 500       | 124750   | 124750   | 124750   |
| 1000      | 499500   | 499500   | 499500   |
| 5000      | 12497500 | 12497500 | 12497500 |



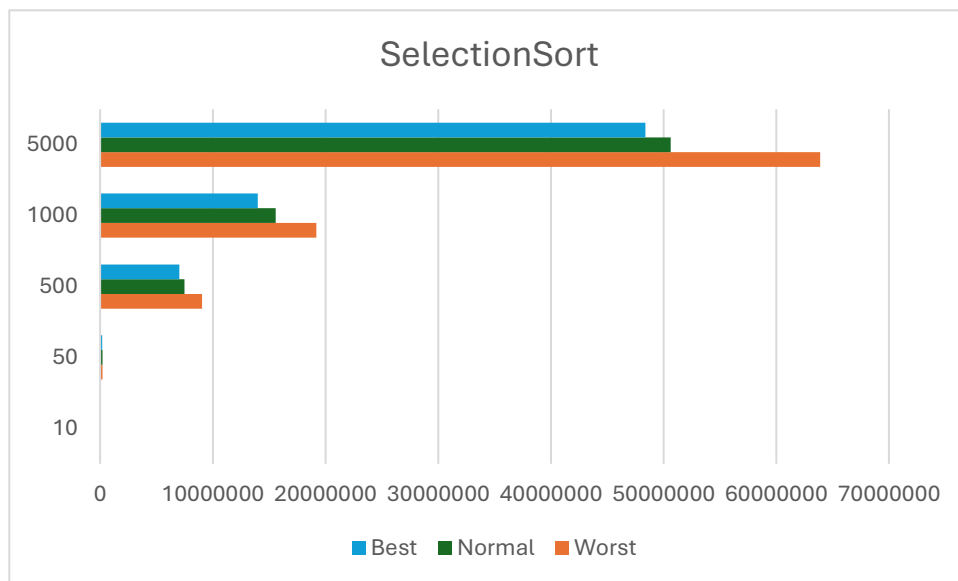
En BubbleSort se puede ver observar que en línea general las comparaciones que se realizan son iguales en todos los casos, mientras que en los swaps y los tiempos existe una diferencia notable, ya que el mejor caso valga la

redundancia es el mejor caso al estar todo ordenado no necesita realizar swaps y en tiempos es rápido el análisis de este, mientras que en el peor, su tiempo de ordenamiento no es muy alto en listas pequeñas, pero tiene un gran escalado para listas con una gran cantidad de valores, y a la hora de realizar swaps siempre será el mismo orden en el que se realiza el recorrido, el único factor que influye es el tamaño de la lista; En el caso promedio, al tener una lista con valores ordenados de manera aleatoria, su tiempo puede variar mucho dependiendo que tan cercana este la lista a el peor caso, pero en swaps siempre será menor que el peor caso.

SelectionSort:

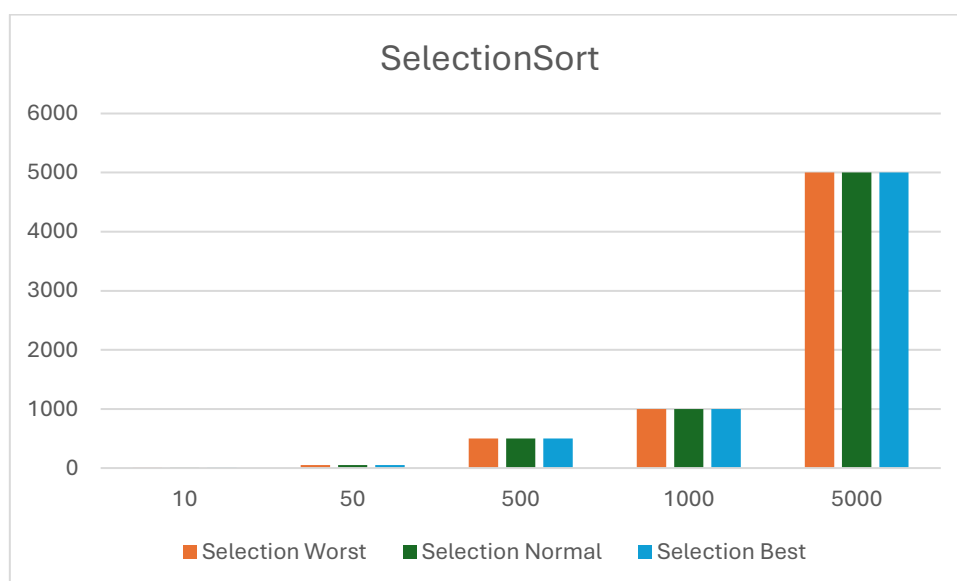
**Tiempos**(En Nanosegundos)

| Contenido | SelectionSort |          |          |
|-----------|---------------|----------|----------|
|           | Worst         | Normal   | Best     |
| 10        | 31300         | 32500    | 41700    |
| 50        | 219700        | 213800   | 190600   |
| 500       | 9039700       | 7473900  | 7018700  |
| 1000      | 19187399      | 15588101 | 14000001 |
| 5000      | 63895499      | 50626300 | 48386700 |



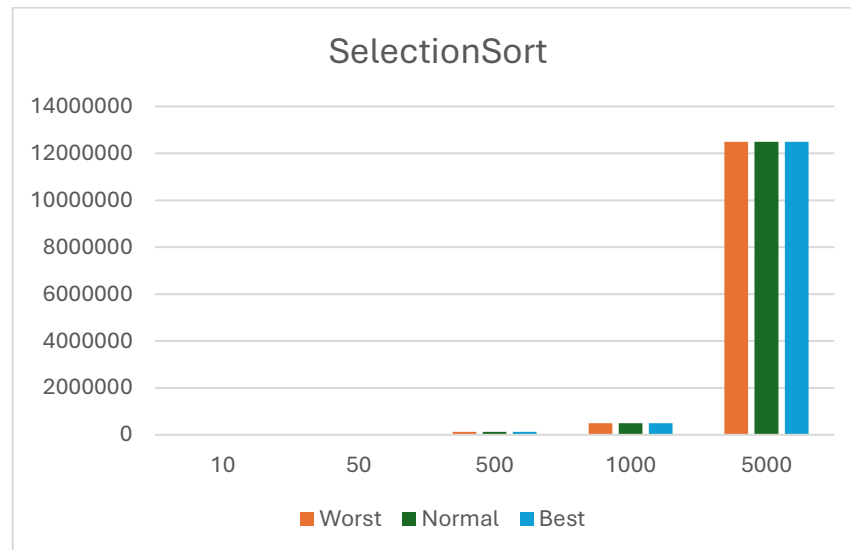
## Swaps

| Contenido | Selection |        |      |
|-----------|-----------|--------|------|
|           | Worst     | Normal | Best |
| 10        | 10        | 10     | 10   |
| 50        | 50        | 50     | 50   |
| 500       | 500       | 500    | 500  |
| 1000      | 1000      | 1000   | 1000 |
| 5000      | 5000      | 5000   | 5000 |



## Comparaciones

|           | Selection |          |          |
|-----------|-----------|----------|----------|
| Contenido | Worst     | Normal   | Best     |
| 10        | 45        | 45       | 45       |
| 50        | 1225      | 1225     | 1225     |
| 500       | 124750    | 124750   | 124750   |
| 1000      | 499500    | 499500   | 499500   |
| 5000      | 12497500  | 12497500 | 12497500 |

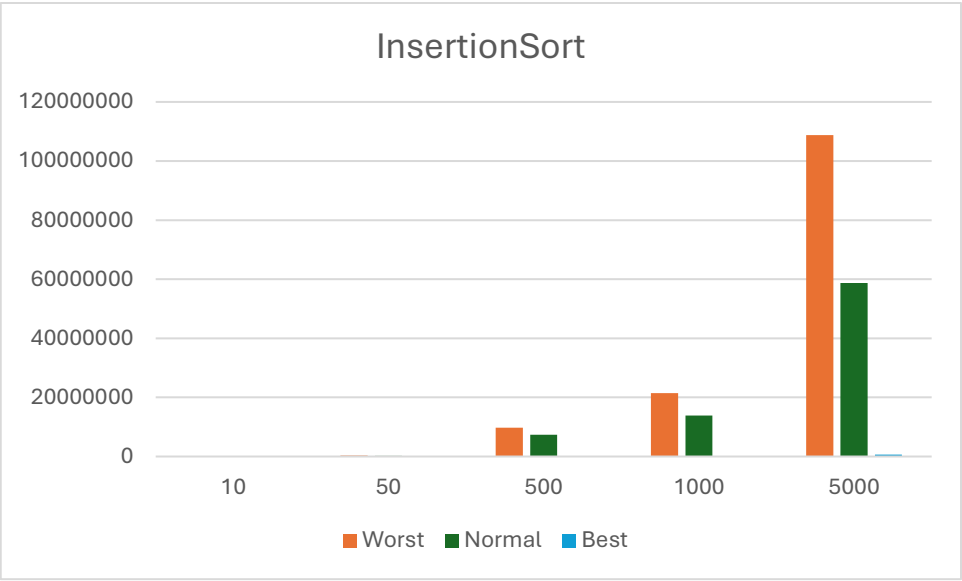


El SelectionSort, en todos los casos, hablando de las comparaciones y los swaps, serán iguales ya que siempre se busca el menor elemento y se recorre toda la lista, para luego posicionarlo, entonces no influye el orden de la lista a en las comparaciones y swaps (cabe aclarar que en el mejor caso el SelectionSort reemplaza el mismo valor y lo mantiene en su posición actual, lo cual se considera como un swap) pero el orden de la lista si influye en los tiempos, ya que en todos los casos al ser listas pequeñas son casi iguales los tiempos, pero a medida que se agranda la cantidad de elementos que están en la lista, mientras mas se aleje el caso promedio del peor, es mas notable la diferencia de lo que se tarda en ordenar la lista.

InsertionSort:

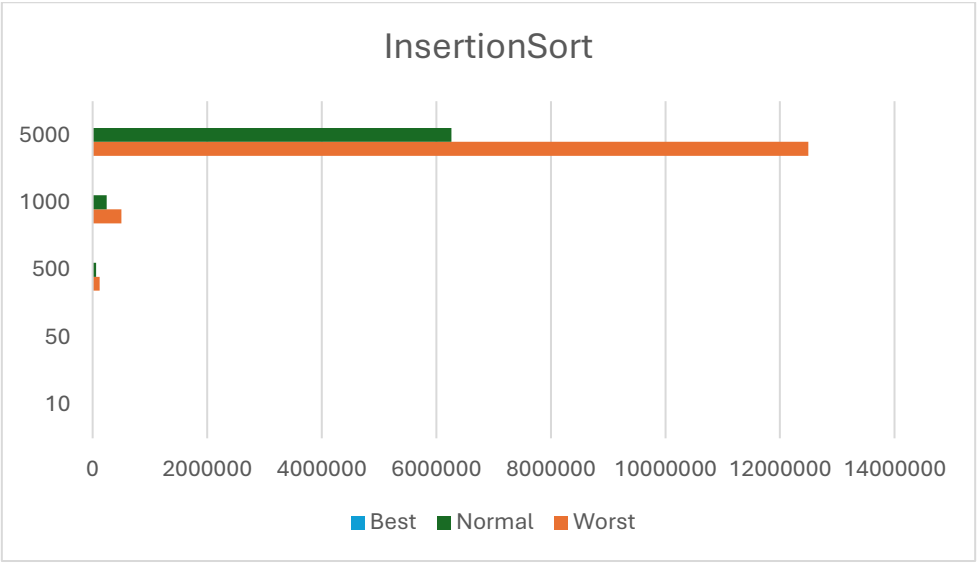
### Tiempos(En Nanosegundos)

|           | InsertionSort |          |        |
|-----------|---------------|----------|--------|
| Contenido | Worst         | Normal   | Best   |
| 10        | 44499         | 22199    | 11800  |
| 50        | 302700        | 177900   | 11400  |
| 500       | 9780199       | 7394500  | 55400  |
| 1000      | 21462899      | 13826501 | 117001 |
| 5000      | 108806200     | 58679700 | 664500 |



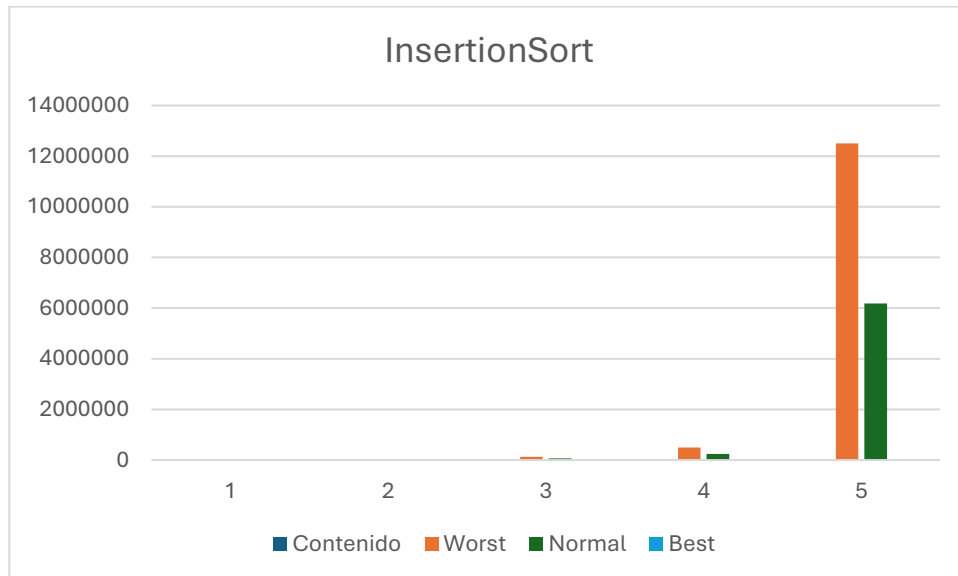
**Swaps**

|           | Insertion |         |      |
|-----------|-----------|---------|------|
| Contenido | Worst     | Normal  | Best |
| 10        | 45        | 23      | 0    |
| 50        | 1225      | 527     | 0    |
| 500       | 124750    | 62342   | 0    |
| 1000      | 499500    | 242439  | 0    |
| 5000      | 12497500  | 6265391 | 0    |



## Comparaciones

| Contenido | Insertion |         |      |
|-----------|-----------|---------|------|
|           | Worst     | Normal  | Best |
| 10        | 45        | 27      | 9    |
| 50        | 1225      | 684     | 49   |
| 500       | 124750    | 62641   | 499  |
| 1000      | 499500    | 246486  | 999  |
| 5000      | 12497500  | 6187544 | 4999 |



El InsertionSort, en el peor caso las comparaciones y los swaps son idénticos, mientras que en caso promedio varía mucho de qué tan aproximado este la lista a lucir como el peor caso o el mejor, y esto se refleja en los tiempos, en el swap y en las comparaciones. Mientras que a diferencia de estos, el mejor caso presenta tiempos muy pequeños para todo tipo de lista, en swaps no se realiza ninguno, y las comparaciones son el tamaño de la lista – 1, ya que considera al primer elemento como si ya estuviera en su posición correcta.

### Ejercicio 3:

Elegí el SelectionSorter, ya que su tiempo de complejidad es  $n^2$ , pero en mis pruebas de tiempo de ejecución, resulto ser mas eficaz en listas grandes o medianas, en comparación con el resto.